

COMPILER DESIGN

[CS-603]

UNIT - 2

SYNTAX ANALYSIS & SYNTAX DIRECTED TRANSLATION

SYLLABUS -

Syntax Analysis: CFGs, Top down parsing, Brute force approach, recursive descent parsing, transformation on the grammar, predictive parsing, bottom up parsing, operator precedence parsing, LR parser (SLR, LALR, LR), Parser generation.

Syntax directed definitions: Construction of Syntax trees, Bottom up evaluation of S-attributed definition, L-attribute definition

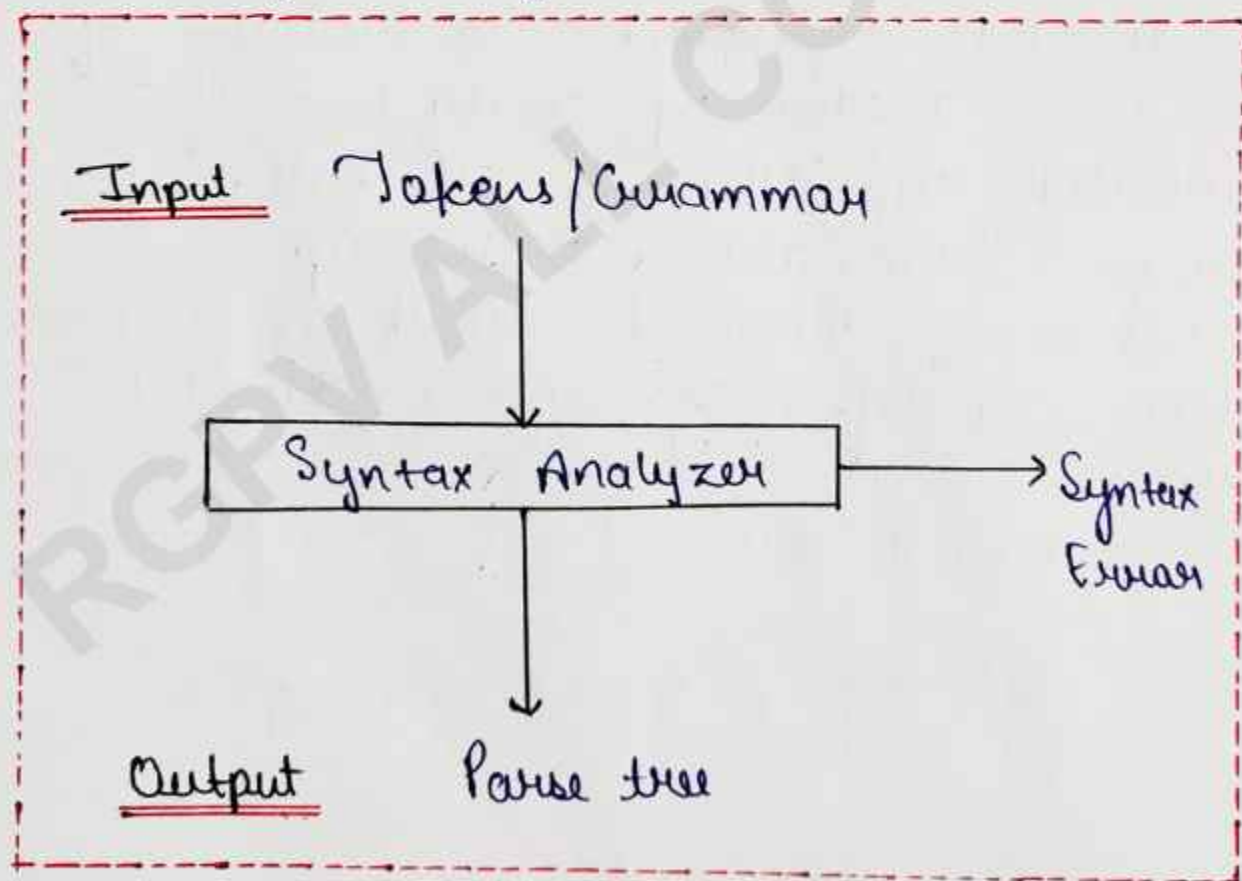
Top-down translation, Bottom up evaluation of inherited attributes Recursive Evaluation, Analysis of syntax directed definition.

INTRODUCTION OF SYNTAX ANALYSIS

Syntax Analysis is the second phase of compiler.

Checking given string is member of given grammar (or) not is known as Syntax Analysis or Parsing.

Syntax Analyzer is a program that takes tokens as input & generate parse tree in the form of output.



In this one more input is included that is grammar.

ROLE OF SYNTAX ANALYZER (PARSER)

- It is a program that takes tokens and CFN as input & verifies tokens are member or not.
- If tokens are member, parser produces parse tree as output. Otherwise syntax error is generated.

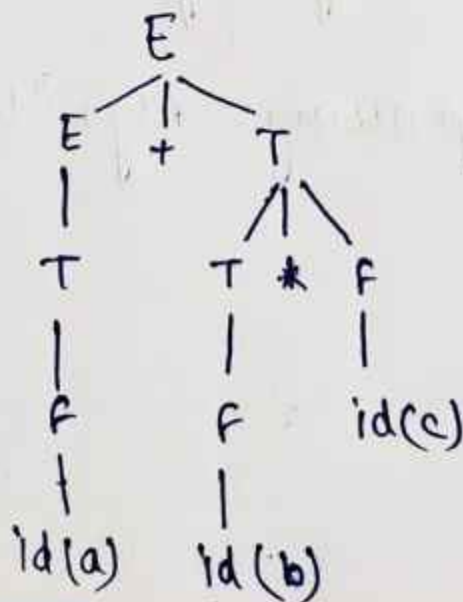
PARSE TREE

A tree structure that shows how a sentence is derived using grammar.

Example:-

Consider a grammar - $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

Expression :- $a + b * c$



Syntax Errors

Errors detected by parser:

- Missing semicolon
- Wrong order of tokens
- Unmatched brackets.

Example:-

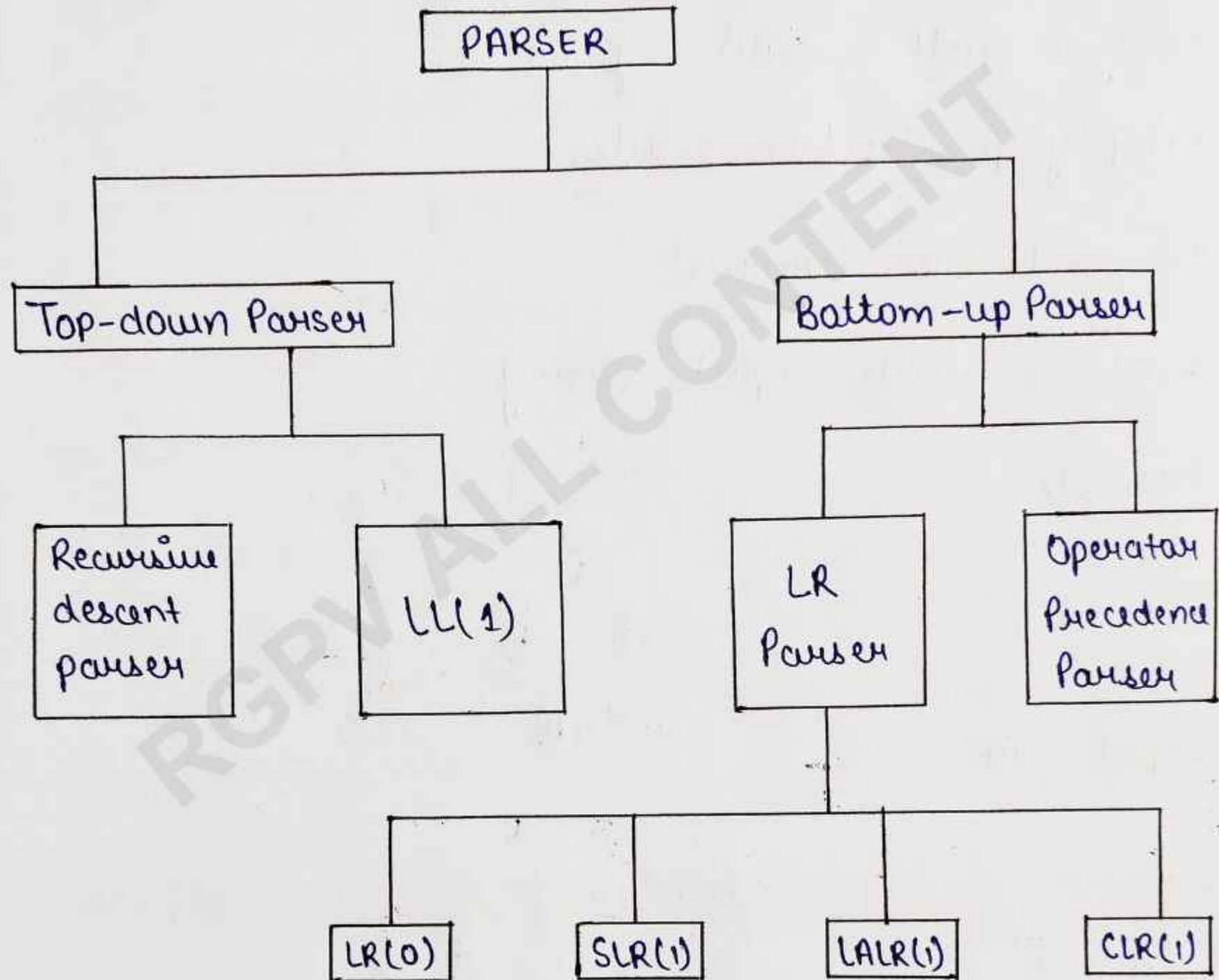
int a = 10 (missing ;)

PARSING ALGORITHM

Parser or syntax Analyzer is a deterministic pushdown automata.

To design parser two types of Algorithm exist known as-

- Bottom-up parsing Algorithm
- Top-down parsing Algorithm



[RGPU, May 2022]

CONTEXT FREE GRAMMAR (CFG)

A Context Free Grammar (CFG) is a set of rules used to describe the syntax of a programming language.

It helps in generating all valid string of a language.

Formal Definition -

A CFG is defined as a 4-tuple -

$$G: (V, T, P, S)$$

Where,

$V \rightarrow$ Set of Non-terminals (S, A, B)

$T \rightarrow$ Set of Terminals ($a, b, +, id$)

$P \rightarrow$ Set of Production rule ($A \rightarrow a$)

$S \rightarrow$ Starting Symbol (S)

Example:

Grammar:

$$S \rightarrow aSb \mid \epsilon$$

This generates string like:

ab

aabb

aaabbb

Derivation in CFL

Process of generating strings from Grammar.

Types:

- **Leftmost Derivation**

Always replace leftmost Non-Terminal.

- **Rightmost Derivation**

Replace rightmost non-terminal.

TRANSFORMATION OF GRAMMARS

Grammar Transformation means converting a grammar into a form that is suitable for parsing (especially top-down parsing).

In simple words:-

Modify grammar to remove problems & make parsing easy.

Why Transformation is Needed?

Because some grammars have problems like:

- Left Recursion
- Ambiguity
- ~~Left~~ Common prefixes.

So we transform grammar to:

- make it efficient
- make it non-backtracking
- make it suitable for LL(1) parser.

Types of Grammar Transformation

1. Elimination of Left Recursion [May, 2024]

$S \rightarrow Sa|b \rightarrow \text{Problem}$

Causes infinite loop in top-down parsing.

Solution $\rightarrow$

$$S \rightarrow bS'$$

$$S' \rightarrow aS' | \epsilon$$

Example:-

$$G: E \rightarrow E+T | T$$

$$T \rightarrow a$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow a$$

2. Left Factoring [Dec, 2024]

When two production have common prefix.

$$S \rightarrow ab|ac|ad$$

Parser gets confused

Solution:-

$$S \rightarrow aS'$$

$$S' \rightarrow b|c|d$$

[Dec, 2024]

BACKTRACKING & NON-BACKTRACKING PARSER

BACKTRACKING PARSER-

A Backtracking parser is a type of parser that:

- Tries one production rule.
- If it fails → goes back (backtrack)
- Then tries another rule.

Try → fail → Go Back → Try Again

Example:-

Grammar:

$S \rightarrow aA \mid bB$

$A \rightarrow b$

$B \rightarrow c$

Input $\rightarrow bc$

Steps:

Try $S \rightarrow aA$

$A \rightarrow b$ (does not match input c)

Backtrack

Try $S \rightarrow bB$

$B \rightarrow c$ (Success)

Problems of Backtracking-

- It is very slow (wastes time)
- Repeats work
- Not efficient for large programs

NON-BACKTRACKING PARSER -

A Non-Backtracking Parser

- Does not go back.
- Uses smart rules to choose correct production
- Predicts correct path in one go.

Example:-

Grammar:

$S \rightarrow aA \mid bB$

$A \rightarrow b$

$B \rightarrow c$

Input $\rightarrow bc$

Steps:

~~Try~~ $S \rightarrow bB$ (Parser directly chooses)

$B \rightarrow c$

(No confusion, No backtracking)

Backtracking Parser	Non-Backtracking Parser
1. Tries one rule, if fails $\rightarrow$ goes back and tries another 2. Uses trial & error approach 3. Slow Execution 4. Repeats work multiple 5. Hard to use for large programs. 6. May lead to Infinite loops sometimes 7. Ex:- Basic Recursive Descent Parser.	1. Chooses correct rule in one step. 2. Uses predictive approach 3. Fast Execution 4. No repetition of work. 5. Suitable for real compilers. 6. No infinite loop problem 7. Ex:- Predictive Parser LL(1).

BRUTE FORCE APPROACH

Brute Force Approach is a parsing method in which:

- The parser tries all possible production rules.
- Until it finds the correct one that matches the input string.

In simple words:

Try every possible way until you get the correct answer.

Top-Down PARSING

[May, 2023 RGPV]

[May, 2024]

Top down parsing starts from the start symbol (S) & tries to generate the input string step-by-step.

In simple words:

Start $\rightarrow$ Expand $\rightarrow$ Match Input.

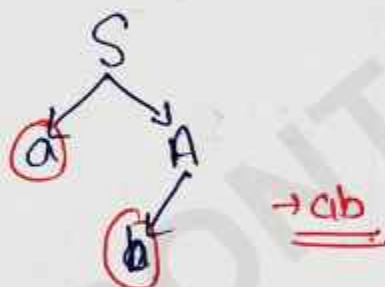
WORKING-

- Begin with start symbol S .
- Apply production rules.
- Expand Non-terminal
- Match with input string.

Example-

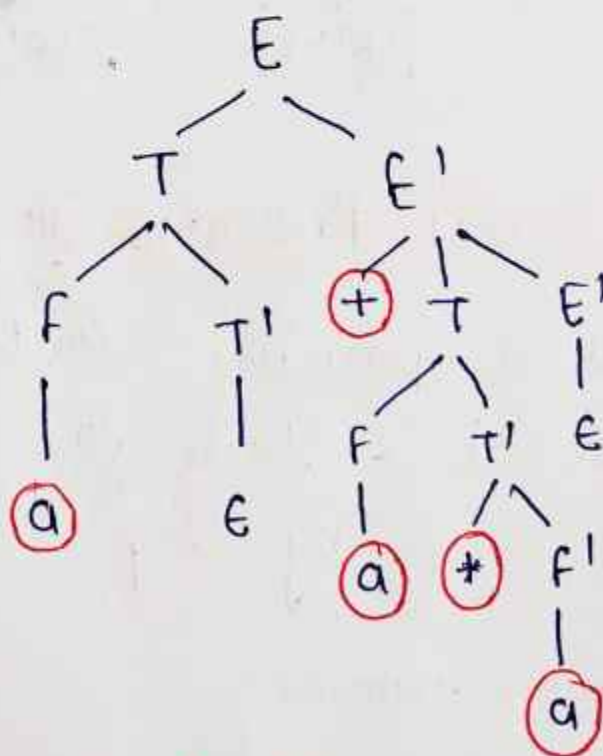
G:
 $S \rightarrow aA$
 $A \rightarrow b$

Input $\rightarrow ab$



G:
 $E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *F \mid \epsilon$
 $F \rightarrow a$

Input String $\rightarrow a + a * a$



$\rightarrow a + a * a$

Types of Top-Down Parsing

1. Recursive Descent Parsing

- Uses Recursive functions
- May use backtracking

2. Predictive Parsing (LL(1))

- No backtracking
- Uses lookahead symbol
- Uses parsing table.

RECURSIVE DESCENT PARSING

[May, 2023]

Recursive Descent Parsing is a type of top-down parsing in which each non-terminal is implemented as a recursive function.

Working Principle -

- Starts from start symbol (S)
- For each non-terminal $\rightarrow$ call a function
- Functions call each other recursively.
- Match input string step-by-step

Example:-

Gr: $S \rightarrow aA$
 $A \rightarrow b$

Input $\rightarrow ab$

Functions:- $S()$ {

 match('a');

 {
 $A()$;
 }

$A()$ {

 match('b');

}

Execution:

call $S()$ $\rightarrow$ match 'a'

call $A()$ $\rightarrow$ match 'b'

Input accepted.

Advantages-

- Simple to implement
- Easy to understand
- Useful for small grammars.

Disadvantages-

- Cannot handle left recursion
- May require backtracking
- Not efficient for complex grammars.

Improved version $\rightarrow$ Predictive Parser.

[RGPV, May 2022]

PREDICTIVE PARSING (OR) LL(1) PARSING

Predictive Parsing is a type of top-down parsing that does not use backtracking.

It predicts the correct production rule using

Current input symbol (lookahead)

Parsing table.

Why called LL(1) Parsing-

L $\rightarrow$ Scan Input from left to Right

L $\rightarrow$ Produce leftmost derivation

1 $\rightarrow$ Use 1 lookahead symbol

Working:-

Uses -

- Input Buffer
- Stack
- Parsing Table

Steps:-

1. PUSH Start Symbol (s) into stack.
2. Read input string
3. Compare stack top with input
4. Use parsing table to select rule
5. Replace Non-terminal with RHS.
6. Repeat until input is accepted.

Predictive Parsing Table -

Table used to decide which rule to apply.

Based on:

- FIRST set
- FOLLOW set.

FIRST Set ^[Dec, 2024] → Set of terminals which appears in the first position on RHS part of grammar.

Rules:-

1) $\text{First } \downarrow \underline{a} b c = \{a\}$

2) $A \rightarrow \underline{a} x$
 $\xrightarrow{\text{terminal value}}$
 $\text{First}(A) = \{a\}$

3) $A \rightarrow Bx$
 $\xrightarrow{\text{Non-terminal value}}$
 $\text{First}(A) = \text{First}(B)$

4) $A \rightarrow Bx$

$$\text{First}(A) = \{\text{First}(B) - \epsilon\} \cup \{\text{First}(x)\}$$

Example:- $S \rightarrow AB$ $\text{First}(B) \rightarrow \{b\}$
 $A \rightarrow a | b$ $\text{First}(A) \rightarrow \{a, b\}$
 $B \rightarrow b$ $\text{First}(S) \rightarrow \{a, \text{First}(A)\} \rightarrow \{a, b\}$

$S \rightarrow AB$ $\text{First}(B) \rightarrow \{d\}$
 $A \rightarrow a | \epsilon$ $\text{First}(A) \rightarrow \{a, \epsilon\}$
 $B \rightarrow d$ $\text{First}(S) \rightarrow \{\text{First}(A) - \epsilon\} \cup \{\text{First}(B)\}$
 $\rightarrow \{(a, \epsilon) - \epsilon\} \cup \{d\}$
 $\rightarrow \{a\} \cup \{d\}$
 $\rightarrow \{a, d\}$

[Dec, 2024]

Follow Set $\rightarrow$ Follow set of a Non-terminal is set of terminals which appears immediately RHS of that Non-terminal in some derivation step.

Rules:

- 1) $\text{Follow}(S) = \{\$ \}$ [Starting Symbol]
- 2) If Grammar is $A \rightarrow aBX$ [$X \neq \epsilon$] $\therefore$ Follow never contains (ϵ)
 $\text{Follow}(B) = \text{First}(X)$
- 3) If Grammar is $A \rightarrow a\underline{B}$ (Last Symbol)
 $\text{Follow}(B) = \text{Follow}(A)$
- 4) If Grammar is $A \rightarrow BX$ [$X \neq \epsilon$]
 $\text{Follow}(B) = \{\text{First}(X) - \epsilon\} \cup \{\text{Follow}(A)\}$

Example:-

$S \rightarrow AaAb \mid BbBa$ $\text{Follow}(S) = \{\$ \}$ $S \rightarrow$ Starting Symbol

$A \rightarrow a$

$\text{Follow}(A) = \{a, b\}$

$B \rightarrow b$

$\text{Follow}(B) = \{b, a\}$

Question:- Consider the following Grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

[RGPV, PYQ]
2023

Eliminate the left Recursion for the grammar & Find First & Follow sets for each Non-Terminal & Also constructed Predictive Parsing Table.

After Eliminating left-recursion the grammar is

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

First():

$$\text{First}(E) = \{ (, id \}$$

$$\text{First}(E') = \{ +, \epsilon \}$$

$$\text{First}(T) = \{ (, id \}$$

$$\text{First}(T') = \{ *, \epsilon \}$$

$$\text{First}(F) = \{ (, id \}$$

Follow():

$$\text{Follow}(E) = \{ \$,) \}$$

$$\text{Follow}(E') = \{ \$,) \}$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

$$\text{Follow}(T') = \{ +, \$,) \}$$

$$\text{Follow}(F) = \{ +, *, \$,) \}$$

Predictive Parsing Table [LL(1)] -

non terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow fT'$			$T \rightarrow fT'$		
T'		$T \rightarrow \epsilon$			$T \rightarrow \epsilon$	$T \rightarrow \epsilon$
f	$f \rightarrow id$			$f \rightarrow (\epsilon)$		

BOTTOM - UP PARSING [RGPV, may 2023] [may, 2024]

- Construct a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top).
- Bottom-up parser using right most derivation in reverse order
- We can think of this process as one of "reducing" a string (w) to the start symbol of a grammar.

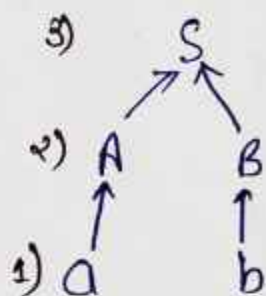
- Bottom-up parser performs parsing by using handle detection.

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$\text{I/P} \rightarrow ab$$



HANDLE

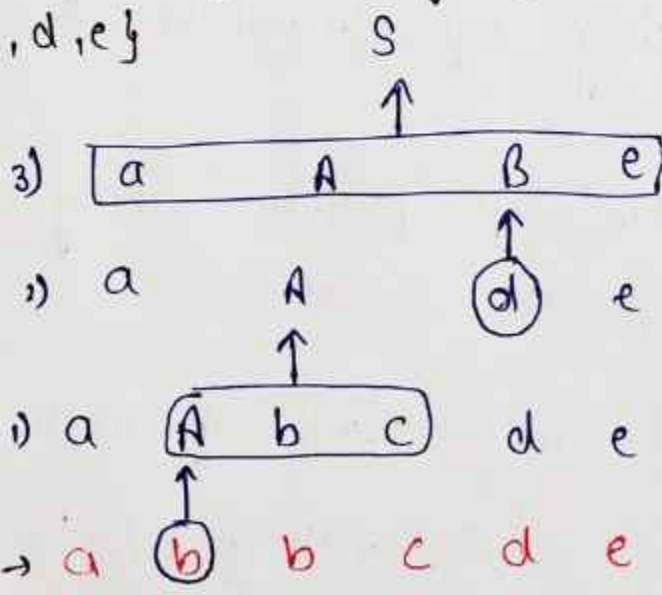
Informally, a "handle" of a string is a sub-string that matches the right side of the production.

How many number of handles detected by bottom-up parser for the following grammar & string. $\{a, b, b, c, d, e\}$

$$S \rightarrow aABe$$

$$A \rightarrow Abc \mid b$$

$$B \rightarrow d$$



String $\rightarrow a \text{ (b) } b \text{ c d e}$

4 Handle Selected { Substring matching }

→ A general type of bottom-up parser is a shift - Reduce Parser.

SHIFT - REDUCE PARSING

A Shift - Reduce Parser is a type of of bottom - up parser.

It works by:

- Shifting input symbols into a stack.
- Reducing them using grammar rules
- until we reach the start symbol.

Definition:-

Shift - Reduce parser is a bottom-up parsing technique that uses a stack to shift input symbols & reduce them step-by-step into the start symbol using grammar rules.

It uses:-

Stack → to store symbols

Input Buffer → to read input.

Operations in Shift-Reduce Parser

1) **Shift** → This involves moving of symbols from input buffer onto the stack.

Example:

Input: $id + id$

Stack: id (after shift)

2) **Reduce** → Replace symbols on stack using grammar rule.
(Replace Rhs with Lhs)

Example:

$T \rightarrow id$ (Rule) / (Grammar)

So

$id \rightarrow T$

3) **Accept** → If only start symbol is present in the stack & the input buffer is empty (ϵ) the parsing action is called accept.

When accept action is obtained, it means successful parsing is done.

4) **Error** → This is the situation in which the parser can neither perform shift action or reduce action & not even accept action.

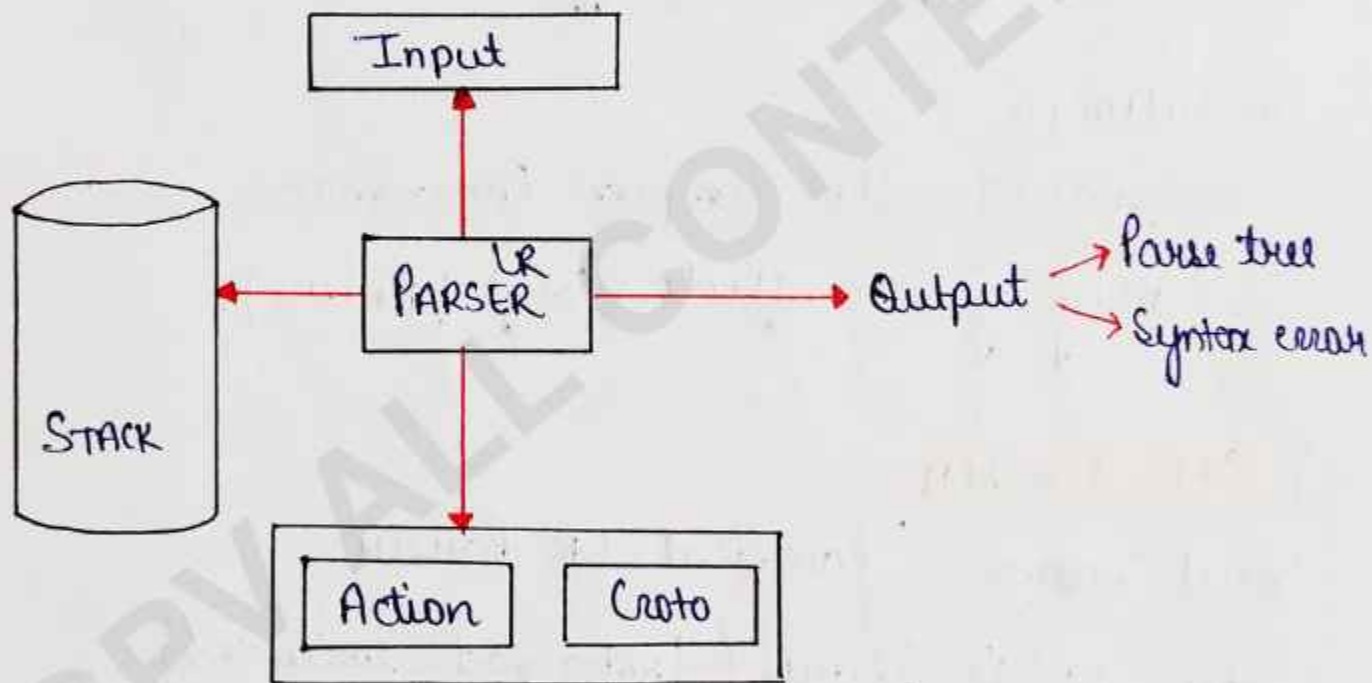
LR - PARSING

LR-Parsing is an advanced bottom-up parsing technique in Computer Design.

LR stands for:

L → Left to Right scan of input

R → Right most Derivation in reverse order.



Uses of LR PARSE

- Handles large class of grammars
- No backtracking needed.
- Detects error early.
- Used in real compilers (like YACC).

Types of LR Parser

1) SLR Parser

- Full Form :- Simple LR Parser.
- Uses LR(0) items & Follow set

Features:

- Easy to Construct
- Small Parsing Table
- Less computational Effort

Limitations:

- Can't handle complex Grammars
- may give conflicts (Shift | Reduce).

2) CLR Parser

- Full Form :- Canonical LR Parser
- Uses LR(1) items (lookahead included)

Features:

- Most Powerful LR Parser
- Handles almost all Context free grammars.

Limitations:

- Very large Parsing Table
- Complex to implement.

3) LALR Parser

- Full Form :- Lookahead LR Parser
- Combines SLR + CLR
- merges similar states of CLR

Features:

- Table size small (like SLR)
- Power almost equal to CLR.
- Efficient

Most commonly used in real compilers.

Conflicts in Shift-Reduce Parser [RUPV, May 2024]

A conflict occurs when the parser cannot decide what action to take at a particular step.

means parser is confused between:

Shift (read next ^{IP} symbol) or

Reduce (apply a grammar rule).

Types of Conflicts

- Shift-Reduce Conflict
- Reduce-Reduce Conflict

1) SHIFT-REDUCE Conflict

A Shift-Reduce Conflict occurs when the parser is confused between: SHIFT | REDUCE?

It happens when:

- Grammar is ambiguous
- operator precedence is not defined.
- Associativity is not clear.

Example:- Consider a Grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

$$I/P \rightarrow id + id * id$$

→ At some point, parser stack contains

$$E + E$$

Next input symbol: *

→ Now parser is confused:

Option-1: Reduce

$$E + E \rightarrow E$$

Option-2: Shift

Shift * to handle higher precedence (*)

→ This is Shift-Reduce Conflict.

This Happens-

Because Grammar does not specify:

* has higher precedence than +.

So parser cannot decide correct order.

How to resolve it?

Method - 1: Operator Precedence

$* > +$

Method - 2: Rewrite Grammar

Unambiguous Grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id.}$

Now, No confusion.

Real-Life Example-

Like solving math:

$2 + 3 * 4$

If precedence is not define-

Solve $2 + 3$ first?

OR $3 * 4$ first?

REDUCE - REDUCE CONFLICT

A Reduce-Reduce Conflict occurs when parser is confused:

which rule should I reduce?

It occurs when-

- Two or more production match same input.
- Grammar is highly ambiguous.

Example:- Grammar:

$A \rightarrow id$

$I/P \rightarrow id$

$B \rightarrow id$

→ Parser situation

Stack id

→ Now parser has 2 choices:

Option 1:

Reduce $id \rightarrow A$

Option 2:

Reduce $id \rightarrow B$

→ Parser can't decide - Reduce / Reduce Conflict.

LR(0) Items

An LR(0) item is a production rule with a dot (.) at some position in the right hand side.

The dot shows how much of the production has been seen (parsed).

LR(0) items are -

$S \rightarrow AB$

$S \rightarrow \cdot AB$

$A \rightarrow \cdot a$

$A \rightarrow a$

$S \rightarrow A \cdot B$

$B \rightarrow \cdot b$

$B \rightarrow b$

$S \rightarrow AB \cdot$

$B \rightarrow b \cdot$

$A \rightarrow a \cdot$

Closure()

Closure is the process of adding more LR(0) items to a given set of items based on non-terminals after the dot (.).

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Closure()

$S \rightarrow \cdot AB$

$A \rightarrow \cdot a$

Closure

$S \rightarrow A \cdot B$

$B \rightarrow \cdot b$

→ Add items only when dot is before non-terminal

→ Add production with dot at start

→ Repeat until no change

GOTO() -

Shift the dot (.) on place to the RHS

LR(0) item

$S \rightarrow A \cdot B$ $\xrightarrow{\text{goto()}}$ $S \rightarrow AB \cdot$

$A \rightarrow \cdot a$ $\xrightarrow{\text{goto()}}$ $A \rightarrow a \cdot$

SLR PARSING

[RUPV, may 2022] [Dec, 2024]
[RUPV, may 2023]

Steps to Construct SLR Parsing Table

- 1) Construct Augmented Grammar
[Add $S' \rightarrow S$]
- 2) Construct LR(0) items DFA by using
closure() and goto().
- 3) Construct Parsing Table

Table has ~~two~~ two parts:

ACTION	GOTO
for terminals	for Non-terminals

Construct the SLR Parsing Table for this grammar-

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Step-1:- Construct Augmented Grammar

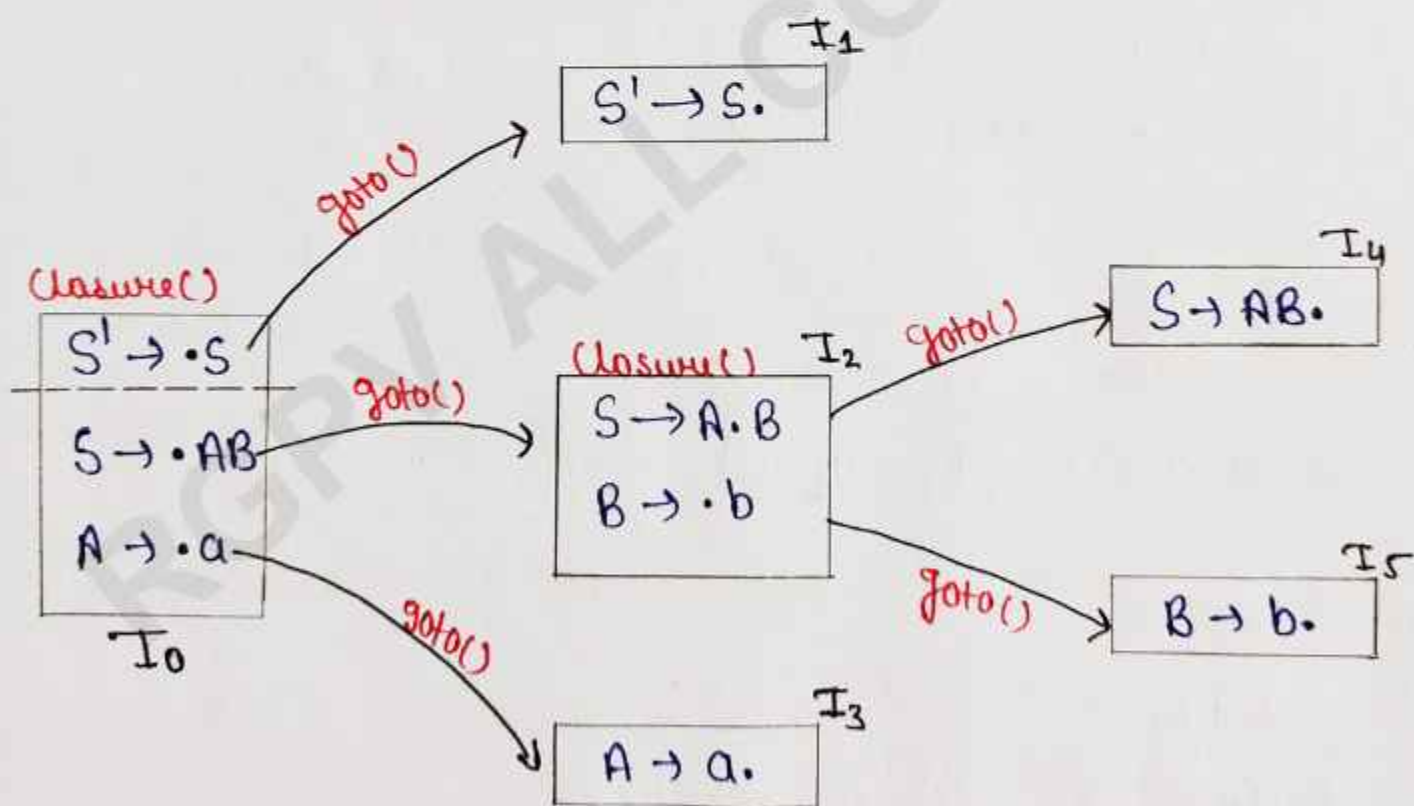
$$S' \rightarrow S \leftarrow \text{start}$$

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Step-2:- Construct LR(0) items DFA



→ 6 States DFA

Step-3 Construct Parsing Table

	a	b	\$	S	A	B
I_0	S_3			1	2	
I_1			Accept			
I_2		S_5				3
H_2 I_3		H_2				
H_1 I_4			H_1			
H_3 I_5			H_3			

At State I_3

$A \rightarrow a.$

$\text{follow}(A) = \{b\} = H_2$ (2)

$S \rightarrow AB$ (1)

$A \rightarrow a$ (2)

$B \rightarrow b$ (3)

At State I_4

$S \rightarrow AB.$

$\text{follow}(S) = \{\$\} = H_1$ (1)

At State I_5

$B \rightarrow b.$

$\text{follow}(B) = \{\$\} = H_3$ (3)

LR(1) Items

An LR(1) item is an extension of an LR(0) item with one extra symbol called lookahead.

$$\text{LR(1) items} = \text{LR(0) item} + \text{lookahead}$$

$$S \rightarrow \cdot AB, \{a\}$$

$$S \rightarrow AB \cdot, \{b\}$$

$$S \rightarrow A \cdot B, \{\$ \}$$

Need of lookahead -

In LR(0), parser may be confused (shift/reduce conflicts).

LR(1) solves this by using 1 lookahead symbol to make better decision.

lookahead symbol of Initial Item -

$$S' \rightarrow \cdot S, \{\$ \}$$

↑
lookahead

$\$ \rightarrow$ End marker.

Closure()

Closure is a function that expands a set of LR(1) items by adding all possible items for non-terminals that appear after the dot ($\cdot$), along with correct lookaheads.

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Closure()

$\left\{ \begin{array}{l} S' \rightarrow \cdot S, \{ \$ \} \\ S \rightarrow \cdot AB, \{ \text{First}(\$) \} = \{ \$ \} \\ A \rightarrow \cdot a, \{ \text{First}(B) \} = \{ b \} \end{array} \right.$ Lookahead

Closure()

$\left\{ \begin{array}{l} S \rightarrow A \cdot B, \{ \$ \} \\ B \rightarrow \cdot b, \{ \$ \} \end{array} \right.$

$E \rightarrow T + E$

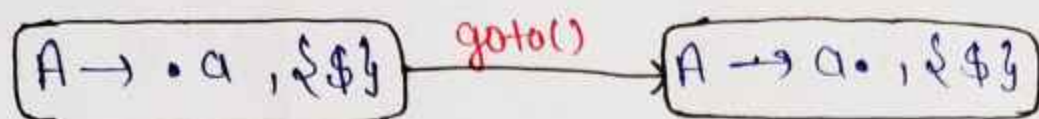
$E \rightarrow T$

$T \rightarrow a$

Closure

$\left\{ \begin{array}{l} E' \rightarrow \cdot E, \{ \$ \} \\ E \rightarrow \cdot T + E, \{ \$ \} \\ E \rightarrow \cdot T, \{ \$ \} \\ T \rightarrow \cdot a, \{ \$, + \} \end{array} \right.$ Lookahead

goto()



CLR(1) PARSING

Steps to Construct CLR Parsing Table

- 1) Construct Augmented Grammar.
[Add $S' \rightarrow S$]
- 2) Construct LR(1) items DFA by using closure() and goto().
- 3) Construct Parsing Table.

Construct the CLR Parsing Table for this grammar

$S \rightarrow Aa$

$S \rightarrow Bb$

$A \rightarrow a$

$B \rightarrow b$

Step-1 Construct Augmented Grammar

$$S' \rightarrow S \leftarrow \text{---}$$

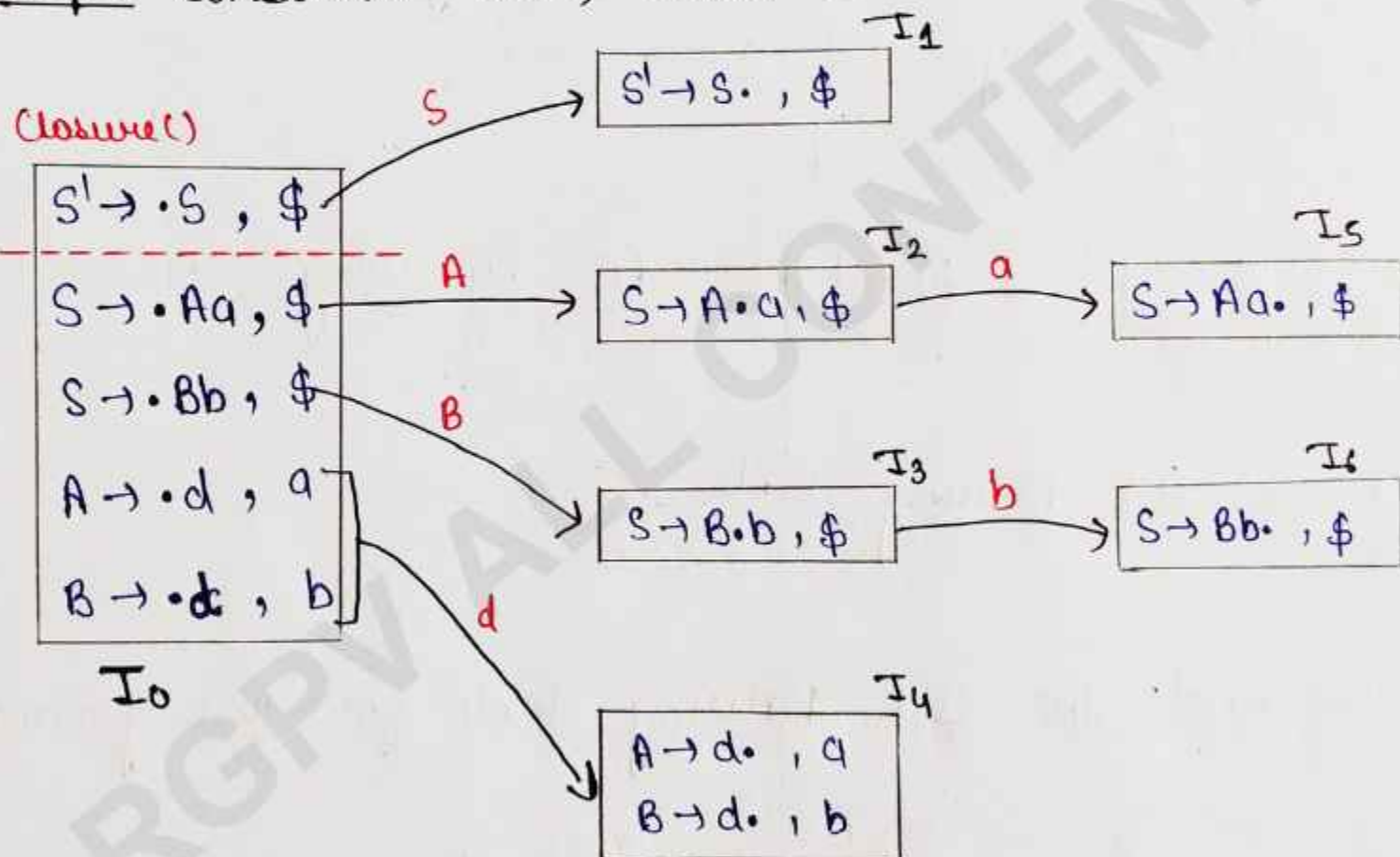
$$S \rightarrow Aa$$

$$S \rightarrow Bb$$

$$A \rightarrow d$$

$$B \rightarrow b$$

Step-2 Construct LR(1) items DFA



→ 7 DFA States.

Step-3 Construct Parsing Table.

	a	b	d	\$	S	A	B
I_0			S_4		1	2	3
I_1				Accept			
I_2	S_5						
I_3		S_6					
I_4	M_3	M_4					
I_5				M_1			
I_6				M_2			

At State I_4

$A \rightarrow d \cdot, a$

$B \rightarrow d \cdot, b$

Lookahead $\rightarrow a, b = M_3, M_4$

$S \rightarrow Aa$ ①

$S \rightarrow Bb$ ②

$A \rightarrow d$ ③

$B \rightarrow d$ ④

At State I_5

$S \rightarrow Aa \cdot, \$$

Lookahead $\rightarrow \$ = M_1$

At State I_6

$S \rightarrow Bb \cdot, \$$

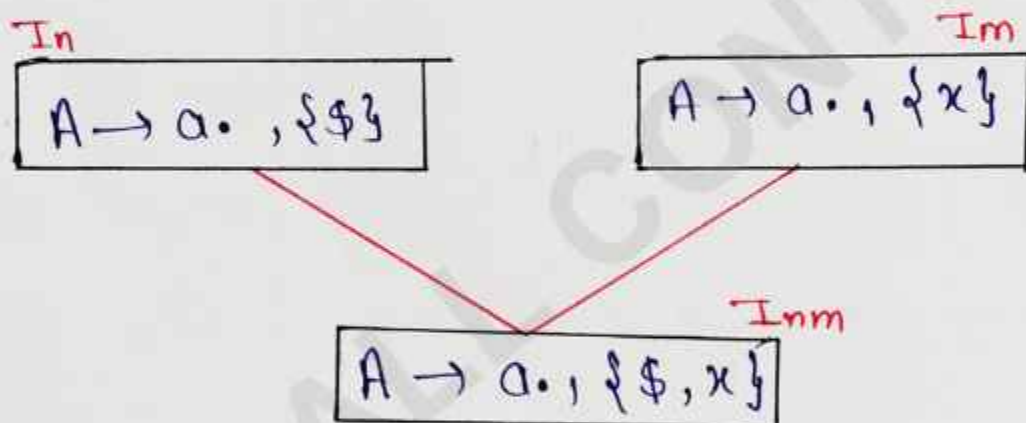
Lookahead $\rightarrow \$ = M_2$

LALR(1) PARSING

[RGPU, May 2024]
[RGPU, June 2025]

Construct LR(1) items DFA using closure() and goto().

Any two (or) more states having same LR(0) part & different lookahead part then merge them into single state & combine all Look-ahead symbol.



Construct a LALR parsing table for this grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

[RGPU, June 2025]

[RGPU, May 2024]

Step-1 Construct Augmented Grammar.

$$E' \rightarrow E \leftarrow \text{---}$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow id$$

Step-2 Construct LR(1) items DFA

we first construct CLR(1) items, then merge states with same LR(0) parts & different lookahead to get LALR(1).

OPERATOR PRECEDENCE PARSING

Operator Precedence Parsing is a type of bottom-up parsing technique used in compiler design to parse expression (like arithmetic expressions) based on the precedence and associativity of operators.

It is used to ~~design~~ decide whether to shift or reduce during parsing.

→ Instead of using a full parsing table (like LR parser), Operator precedence parsing uses precedence relations between terminals to guide parsing decisions.

These relation are:

- $\prec \rightarrow$ yields precedence
- $= \rightarrow$ equal precedence
- $\succ \rightarrow$ takes precedence

Operator Precedence Relation-

for two operators a and b.

1. $a \prec b$ (a yields precedence to b)
→ SHIFT (push b onto stack)

2. $a = \cdot b$ (equal precedence)
 → used for grouping (like parenthesis)

3. $a \rightarrow b$ (a takes precedence over b)
 → Reduce (apply production rule).

Consider a grammar:

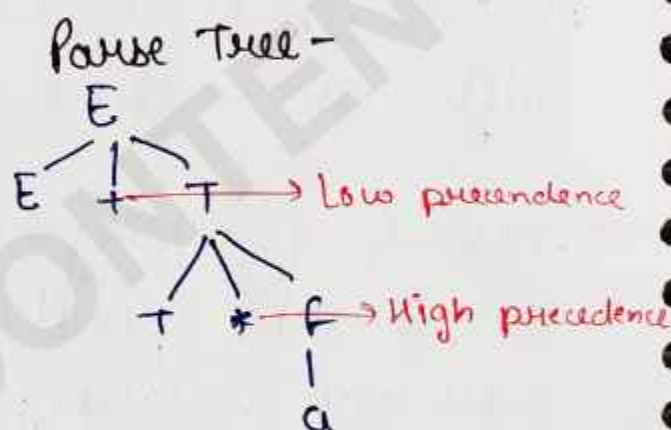
$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow a$$

	a	+	*	\$
a		$\rightarrow$	$\rightarrow$	$\rightarrow$
+	$\times$	$\rightarrow$	$\leftarrow$	$\rightarrow$
*	$\times$	$\rightarrow$	$\rightarrow$	$\rightarrow$
\$	$\leftarrow$	$\leftarrow$	$\leftarrow$	

Operator precedence parsing Table



1) * high precedence
 + low precedence

2) + left Associative
 * left Associative.

3) Identity & terminal have more precedence.

4) \$ have low precedence.

SYNTAX DIRECTED TRANSLATION

Every Context free Grammar production is associated with set of semantic rules.

Semantic rules are attribute computation rules in annotated parse tree.

$$\text{SDT} = \text{CFG} + \text{Semantic rules}$$

Components-

- 1) CFG:- Defines the structure of the language.
- 2) Semantic Rules:- Attached to grammar production to perform task like -
 - Expression evaluation
 - Type checking
 - Intermediate code generation

Example:-

Grammar -

$$E \rightarrow E1 + T$$

$$E \rightarrow T$$

$$T \rightarrow \text{num}$$

Syntax Directed Translation-

$E \rightarrow E1 + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow num \quad \{ T.val = num.val \}$

Semantic rules.

Here,

- $.val$ is an attribute storing value
- Rules compute Expression result.

SDT Advantages-

- 1) SDT is used to construct Semantic Analysis.
- 2) SDT is used to construct Intermediate Code generation.
- 3) SDT is used to construct Symbol Table.
- 4) SDT is used to construct syntax Tree.

Ques. Find E.val for the expression $3 \odot 3 \& 5$ after the evaluation of the following SDT. (IMP)

$$E \rightarrow E1 \odot T \quad \{E.val = E1.val * T.val\}$$

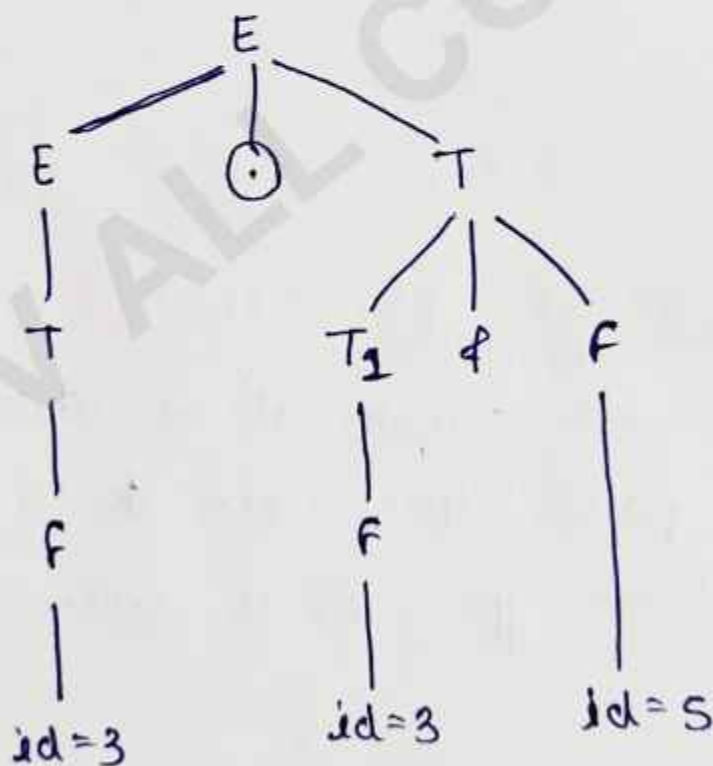
$$E \rightarrow T \quad \{E.val = T.val\}$$

$$T \rightarrow T1 \& F \quad \{T.val = T1.val + F.val\}$$

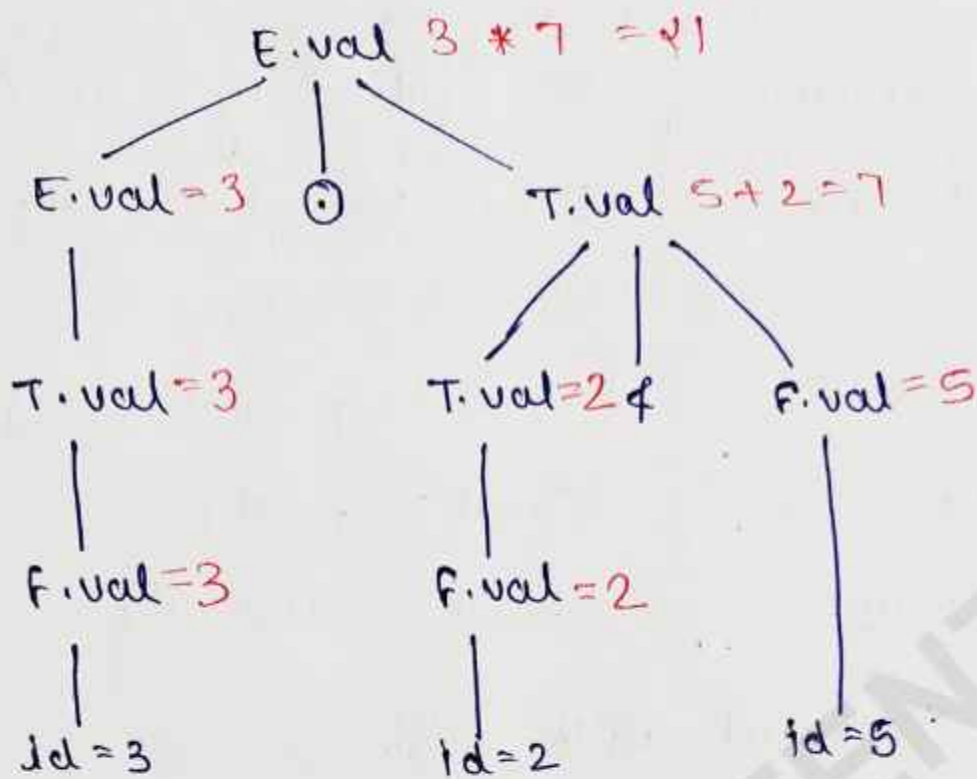
$$T \rightarrow F \quad \{T.val = F.val\}$$

$$F \rightarrow num \quad \{F.val = num.val\}$$

Step-1 Construct Parse Tree. -



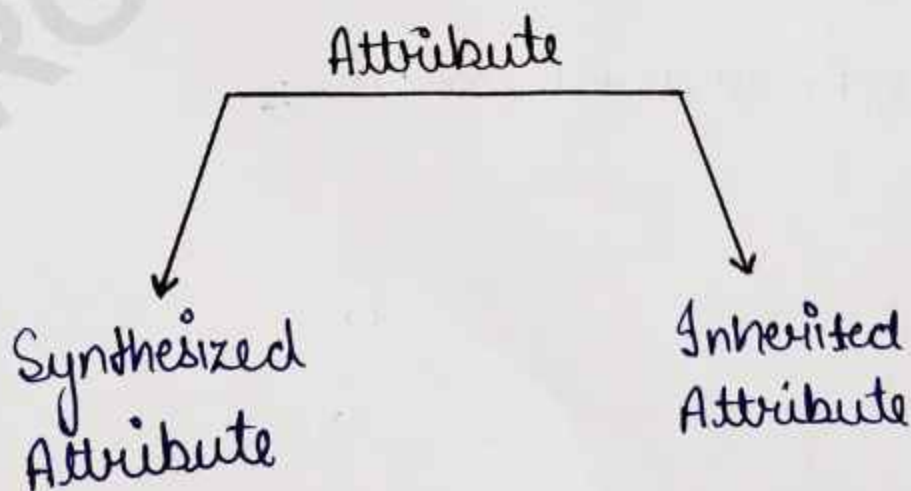
Step-2 Construct Annotated Parse tree -



$E.val = 21$

ATTRIBUTE [May, 2024] [Dec, 2024] [June, 2025]

Attributes are extra information attached to Symbols in a parse tree, used to store details like data type, value, memory location etc.

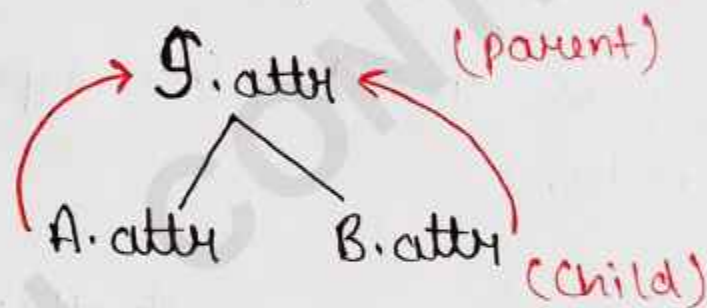


1) SYNTHESIZED ATTRIBUTE -

A Synthesized attribute is an attribute whose value is computed from the attributes of its child nodes in the parse tree.

- Information flows upward in the parse tree.
- Used mainly in bottom-up parsing (like LR parser).
- Very common in evaluating expressions.

$S \rightarrow AB$



Example:

Consider Expression:

$E \rightarrow E1 + T$

Let's define:

$E.value \rightarrow E1.value + T.value$

Here:

- $E1.value$ and $T.value$ come from child nodes.
- $E.value$ is computed using them.

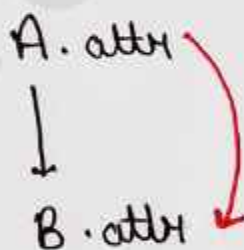
So, value is synthesized attribute.

v) INHERITED ATTRIBUTE -

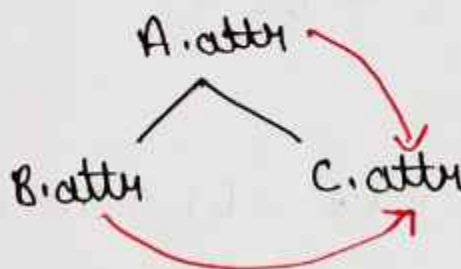
An inherited attribute is an attribute whose value is computed using the attributes of the parent node or sibling nodes in the parse tree.

- Information flows downward or sideways in the parse tree.
- Used mainly in top-down parsing (like recursive descent parser).
- Helps pass context (like type, scope, etc.) to child nodes.

$A \rightarrow B$



$A \rightarrow BC$



Not Allowed.

TYPES OF SYNTAX DIRECTED DEFINITIONS

S-ATTRIBUTE DEFINITION-

An S-Attributed definition is a type of syntax-directed definition (SDD) in which all attributes are synthesized attributes only. No, inherited attributes are used.

Characteristics-

- Used only synthesized attributes
- Information flows bottom $\rightarrow$ up
- Suitable for bottom-up parsing (LR parser)
- Easy to implement.

L-ATTRIBUTE DEFINITION- [June, 2025]

An L-Attribute definition is a type of SDD in which attributes can be both synthesized & inherited, but with a restriction:

The inherited attribute of a symbol can depend only on:

- Parent's attribute
- Left sibling's attributes (not right sibling)

CONSTRUCTION OF SYNTAX TREE-

A Syntax tree is a simplified tree representation of the source program that shows the hierarchical structure of expressions.

- It is derived from the parse tree, but:
- Removes unnecessary grammar symbols.
- Keeps only essential information.

Consider a Grammar:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

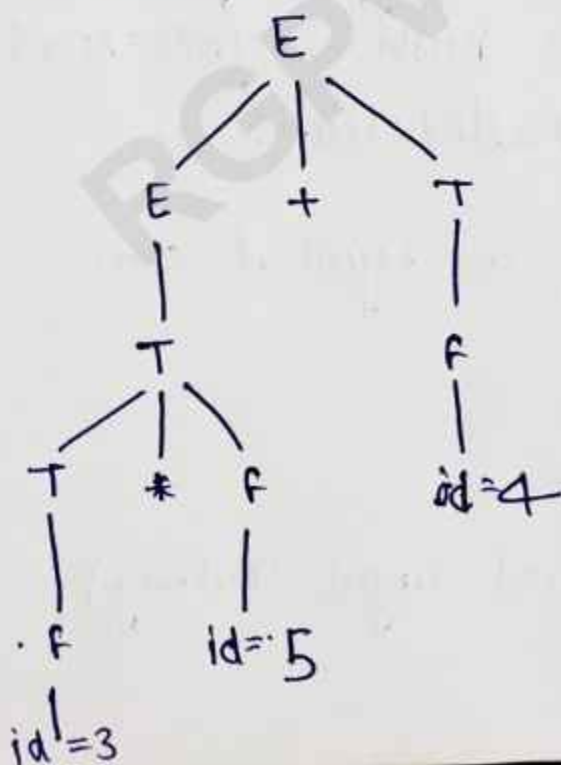
$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow id$$

Construct syntax tree for an expression: ~~3*5*5~~ 3*5+4

Parse Tree



Syntax Tree

